



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Solución de ejercicios

Teoría, diseño de arquitecturas y detección de anomalías

Autor

Armando Arredondo Valle

Maestría en Inteligencia Artificial

Deep Learning

Pontificia Universidad Católica de Chile

Septiembre de 2026

1. Preguntas de teoría

1.1 Invarianza al reescalado y regularización L2

Considere una red sin sesgos definida por $f(\mathbf{x}) = W_2 \text{ReLU}(W_1 \mathbf{x})$. Para algún $\alpha > 0$, se reemplaza W_1 por αW_1 y W_2 por W_2/α . ¿Cambian las predicciones de la red? ¿Cambia el efecto de una regularización L2 aplicada a ambas matrices? Justifique.

Las predicciones no cambian. La ReLU es positivamente homogénea: para todo $\alpha > 0$ se cumple $\text{ReLU}(\alpha z) = \alpha \text{ReLU}(z)$, ya que la multiplicación por una constante positiva no altera el signo de z . Por lo tanto:

$$\frac{W_2}{\alpha} \text{ReLU}(\alpha W_1 \mathbf{x}) = \frac{W_2}{\alpha} \alpha \text{ReLU}(W_1 \mathbf{x}) = W_2 \text{ReLU}(W_1 \mathbf{x}) = f(\mathbf{x}), \quad (1)$$

es decir, la red reescalada calcula exactamente la misma función para toda entrada.

El efecto de la regularización L2 sí cambia. La penalización pasa de $\|W_1\|_F^2 + \|W_2\|_F^2$ a:

$$\alpha^2 \|W_1\|_F^2 + \frac{1}{\alpha^2} \|W_2\|_F^2, \quad (2)$$

que depende de α aunque la función representada sea idéntica. En consecuencia, la regularización L2 no es invariante ante esta reparametrización: entre infinitas redes que calculan la misma función, L2 asigna penalizaciones distintas y prefiere la configuración de normas balanceadas. Esto ilustra que L2 penaliza los parámetros y no directamente la función que la red computa.

Desarrollo: el α que minimiza la penalización. Nótese que $\alpha > 0$ no se despeja, sino que es hipótesis del enunciado, necesaria porque la homogeneidad $\text{ReLU}(\alpha z) = \alpha \text{ReLU}(z)$ solo vale para constantes positivas. Lo que sí puede despejarse es el α óptimo. Escribiendo la penalización como función de α , con $a = \|W_1\|_F^2$ y $b = \|W_2\|_F^2$:

$$g(\alpha) = \alpha^2 a + \frac{b}{\alpha^2}, \quad \alpha > 0. \quad (3)$$

Derivando e igualando a cero:

$$g'(\alpha) = 2\alpha a - \frac{2b}{\alpha^3} = 0 \implies \alpha^4 = \frac{b}{a} \implies \alpha^* = \left(\frac{b}{a}\right)^{1/4} = \sqrt{\frac{\|W_2\|_F}{\|W_1\|_F}}, \quad (4)$$

donde en el segundo paso se multiplicó por $\alpha^3/2$ (válido pues $\alpha > 0$) y en el despeje final se tomó la única raíz real positiva, que es la admisible en el dominio. Se trata de un mínimo porque $g''(\alpha) = 2a + 6b/\alpha^4 > 0$ (la función es convexa en $\alpha > 0$). Sustituyendo α^* se verifica el balance de normas:

$$\|\alpha^* W_1\|_F^2 = (\alpha^*)^2 a = \sqrt{ab} \quad \text{y} \quad \left\| \frac{W_2}{\alpha^*} \right\|_F^2 = \frac{b}{(\alpha^*)^2} = \sqrt{ab}, \quad (5)$$

es decir, ambas capas quedan con igual norma y la penalización mínima vale $g(\alpha^*) = 2\|W_1\|_F\|W_2\|_F$. Alternativamente, sin cálculo diferencial, la desigualdad aritmético-geométrica da directamente $\alpha^2 a + b/\alpha^2 \geq 2\sqrt{ab}$, con igualdad si y solo si $\alpha^2 a = b/\alpha^2$, la misma condición de normas balanceadas.

1.2 Permutación aleatoria de píxeles: CNN vs. MLP

Todas las imágenes de un conjunto de datos son transformadas utilizando la misma permutación aleatoria de sus píxeles, aplicada tanto en entrenamiento como en evaluación. ¿Qué modelo debería verse más afectado, una CNN o un MLP? Explique por qué, considerando los supuestos que incorpora cada arquitectura.

La CNN se ve más afectada; el MLP no se ve afectado.

El **MLP** trata la entrada como un vector plano sin estructura: no incorpora ningún supuesto sobre el orden de las componentes. Permutar los píxeles equivale a permutar las columnas de la matriz de pesos de la primera capa, de modo que para cada solución del problema original existe una solución exactamente equivalente en el problema permutado. Su clase de hipótesis es invariante al reordenamiento de la entrada, por lo que el MLP puede aprender igual de bien en ambos casos.

La **CNN**, en cambio, incorpora dos supuestos arquitectónicos (sesgos inductivos) sobre la estructura espacial de la imagen: *localidad* (los píxeles cercanos están correlacionados y forman patrones locales como bordes y texturas, capturables con filtros pequeños) y *equivarianza a la traslación* (un mismo patrón puede aparecer en cualquier posición, lo que justifica compartir los pesos del filtro por toda la imagen, junto con el *pooling* que agrega información de vecindarios). Una permutación aleatoria destruye precisamente esa estructura: los píxeles vecinos dejan de tener relación entre sí, de modo que los filtros locales y el *pooling* ya no capturan patrones significativos. Aunque la información sigue presente (no se pierde nada), la arquitectura ya no puede explotarla, y la CNN pierde justamente la ventaja que la hace superior en imágenes naturales.

1.3 Dropout y valor esperado de la predicción

En la formulación original de Dropout, durante la inferencia se multiplican las activaciones por $1 - p$ para conservar el valor esperado que tenían durante el entrenamiento. Explique por qué igualar el valor esperado de las activaciones intermedias no implica necesariamente igualar el valor esperado de la predicción final. Considere en su respuesta la presencia de transformaciones no lineales.

El escalamiento por $1 - p$ garantiza la igualdad del valor esperado a la entrada de cada capa: si $m \sim \text{Bernoulli}(1 - p)$ es la máscara, entonces $\mathbb{E}[m \cdot a] = (1 - p)a$. Sin embargo, esa activación pasa luego por transformaciones no lineales g (ReLU, sigmoide, softmax), y en general la esperanza no conmuta con una función no lineal:

$$\mathbb{E}[g(z)] \neq g(\mathbb{E}[z]). \quad (6)$$

(Para funciones convexas o cóncavas, la desigualdad de Jensen indica incluso la dirección del sesgo.)

Durante el entrenamiento, la predicción es una variable aleatoria $f(\mathbf{x}, m)$ que depende de la máscara, y lo que se quisiera reproducir en inferencia es el promedio del ensamble $\mathbb{E}_m[f(\mathbf{x}, m)]$, tomado sobre exponencialmente muchas subredes. La regla de escalar las activaciones introduce la esperanza *dentro* de las no linealidades, capa por capa; en cada capa se comete un error de aproximación que además se propaga y se compone a través de la red. Por eso igualar el valor esperado de las activaciones intermedias no garantiza igualar el valor esperado de la salida: la regla de inferencia de Dropout es solo una aproximación al verdadero promedio del ensamble, que sería exacta únicamente si la red fuera lineal.

1.4 Convolución de 7×7 vs. tres convoluciones de 3×3

Considere una CNN en que todas las capas tienen C canales, utilizan stride 1 y no incluyen sesgo. Compare el uso de una única convolución de 7×7 con una secuencia de tres convoluciones de 3×3 . Analice el campo receptivo, el número de parámetros y la capacidad de representar transformaciones no lineales.

Tabla 1: Comparación entre una convolución de 7×7 y tres de 3×3 .

Criterio	Una conv. 7×7	Tres conv. 3×3
Campo receptivo	7×7	7×7
Parámetros	$7 \cdot 7 \cdot C \cdot C = 49C^2$	$3 \cdot (3 \cdot 3 \cdot C \cdot C) = 27C^2$
No linealidades	1	3

Campo receptivo. Es idéntico. Con stride 1, cada convolución de 3×3 adicional expande el campo receptivo en 2 píxeles por lado: $3 \rightarrow 5 \rightarrow 7$. Ambas alternativas permiten que cada salida dependa de una vecindad de 7×7 de la entrada.

Número de parámetros. La secuencia de 3×3 usa $27C^2$ parámetros frente a $49C^2$ de la convolución única, una reducción cercana al 45 %, con el consiguiente ahorro de memoria y cómputo y un menor riesgo de sobreajuste.

Capacidad no lineal. Si entre las convoluciones se intercalan activaciones (ReLU), la secuencia de tres capas compone tres transformaciones no lineales sobre el mismo campo receptivo, dando lugar a una función más expresiva y “más profunda”; la convolución de 7×7 aplica una única transformación lineal seguida de una sola no linealidad. Cabe notar que, sin activaciones intermedias, las tres convoluciones de 3×3 colapsarían en una única transformación lineal equivalente a una de 7×7 (con rango restringido), por lo que la ventaja expresiva depende de las no linealidades.

Conclusión. Apilar convoluciones de 3×3 domina en ambos ejes (menos parámetros y mayor expresividad para el mismo campo receptivo); este es el principio de diseño introducido por VGG.

2. Diseño de arquitecturas

2.1 Estimación de precios en comercio electrónico

Una plataforma de comercio electrónico desea estimar el precio de publicación de sus productos. Para cada producto dispone de variables numéricas, variables categóricas de alta cardinalidad (marca, vendedor) y un conjunto de etiquetas descriptivas de número variable; en operación pueden aparecer marcas, vendedores o etiquetas no vistas en entrenamiento. Proponga un modelo de Deep Learning: cómo representaría cada tipo de variable, cómo obtendría una entrada de tamaño fijo a partir de las etiquetas y cómo combinaría las representaciones para producir la estimación final.

Representación de cada tipo de variable.

- *Variables numéricas:* se normalizan (estandarización; transformación logarítmica para las de distribución sesgada) y se ingresan directamente al modelo.
- *Categóricas de alta cardinalidad (marca, vendedor):* se representan con *embeddings* aprendidos, una tabla que asigna a cada categoría un vector denso de dimensión moderada. La codificación one-hot sería inviable por la cardinalidad y no capturaría

similitud entre categorías (marcas de gama parecida quedan cercanas en el espacio de embeddings).

- *Categorías nuevas en operación*: se reserva un token *UNK* (desconocido) por campo, cuyo embedding se entrena reemplazando aleatoriamente una fracción de las categorías durante el entrenamiento; alternatively puede usarse *feature hashing*, que mapea cualquier cadena a un número fijo de buckets de embedding.

Entrada de tamaño fijo desde las etiquetas. Cada etiqueta se embebe con una misma tabla de embeddings y el conjunto resultante $\{\mathbf{e}_1, \dots, \mathbf{e}_k\}$ (con k variable) se agrega mediante una operación invariante al orden e independiente de k : promedio o suma (al estilo *Deep Sets*) o, preferentemente, *attention pooling*, que aprende un peso por etiqueta y pondera las más informativas. El resultado es un vector de dimensión fija sin importar cuántas etiquetas tenga el producto. Las etiquetas nuevas se tratan con el mismo mecanismo UNK o hashing (o codificando la etiqueta a nivel de subpalabras para generalizar a partir de su texto).

Combinación y salida. Se concatenan las características numéricas, el embedding de marca, el embedding de vendedor y el vector agregado de etiquetas, y el vector resultante se procesa con un MLP cuya salida es un escalar. El modelo se entrena como regresión (pérdida MSE o Huber), idealmente prediciendo $\log(\text{precio})$, dado que los precios presentan una distribución fuertemente sesgada.

2.2 Clasificación de severidad de accidentes con múltiples fotografías

Se desea clasificar la gravedad de un accidente vehicular utilizando las fotografías tomadas en el lugar. Cada accidente puede tener un número diferente de imágenes, las fotografías no siguen un orden determinado y no todas entregan información igualmente relevante. Proponga una arquitectura: cómo procesaría las fotografías, cómo construiría una representación única del accidente independiente del número y del orden de las imágenes, y qué salida produciría el modelo.

El problema tiene la estructura de *multiple instance learning* sobre un conjunto (no una secuencia) de imágenes, y se resuelve en tres etapas:

1. **Procesamiento por fotografía.** Un mismo encoder convolucional (una CNN o un ViT preentrenado en ImageNet, con pesos compartidos entre todas las fotos) convierte cada imagen i en un vector de características $\mathbf{z}_i$.
2. **Representación única del accidente.** Los embeddings $\{\mathbf{z}_1, \dots, \mathbf{z}_N\}$ se agregan con una operación invariante a permutaciones e independiente de N . La opción más

adecuada es *attention pooling*: el modelo aprende un peso α_i por imagen y produce $\mathbf{z} = \sum_i \alpha_i \mathbf{z}_i$. Esto responde directamente al hecho de que no todas las fotos son igualmente relevantes: las poco informativas reciben peso bajo. (El promedio o el máximo también son agregaciones válidas, pero tratan todas las imágenes por igual o consideran solo la más activada.) Al no depender ni del orden ni de la cantidad de imágenes, se satisfacen ambos requisitos del enunciado.

3. **Salida.** Una capa densa con softmax sobre las clases de severidad, entrenada con entropía cruzada. Dado que la severidad es una variable ordinal (leve < moderada < grave), una alternativa razonable es una formulación de regresión ordinal, que penaliza más los errores entre clases lejanas.

3. Detección de anomalías en piezas mecánicas

Una empresa dispone de una gran cantidad de fotografías de piezas que superaron todos los controles de calidad. Los defectos son poco frecuentes, muy diversos y cambian con los materiales y el proceso, por lo que no existe un conjunto representativo de imágenes defectuosas ni categorías de falla definidas de antemano. Se desea un sistema que reciba la fotografía de una nueva pieza e indique si su apariencia se aleja de lo que el modelo considera normal.

3.1 Formulación del problema de aprendizaje

Dado que solo se dispone de imágenes de piezas normales y que los defectos son raros, diversos y cambiantes (lo que hace inviable un clasificador supervisado con categorías de falla), el problema se formula como **detección de anomalías** o clasificación *one-class*: modelar la distribución de la apariencia normal y señalar como sospechoso todo lo que se desvíe de ella.

- *Regularidades a aprender*: la textura, la geometría, el color y la estructura típicos de las piezas que superaron el control de calidad, es decir, “cómo se ve una pieza buena”.
- *Señal de entrenamiento*: proviene de las propias imágenes, sin etiquetas externas; el objetivo de aprendizaje se construye a partir de la entrada (por ejemplo, reconstruirla).
- *Paradigma*: aprendizaje **no supervisado** (en su variante auto-supervisada), del tipo *one-class learning*.

3.2 Modelo, entrenamiento y prevención de soluciones triviales

Modelo. Un **autoencoder convolucional**: un encoder CNN comprime la imagen a un código latente de baja dimensión y un decoder la reconstruye.

- *Entrada*: la fotografía de la pieza, normalizada (opcionalmente procesada por parches, lo que además permite localizar el defecto).
- *Salida*: la reconstrucción $\hat{\mathbf{x}}$ de la misma imagen.
- *Pérdida*: el error de reconstrucción, por ejemplo el MSE por píxel $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$, o una pérdida basada en SSIM, que captura mejor la estructura local que los píxeles individuales.

Cómo evitar una solución inútil. El riesgo central es que el modelo aprenda la función identidad: si tiene capacidad suficiente para copiar cualquier imagen, también reconstruirá perfectamente las piezas defectuosas y el error de reconstrucción dejará de discriminar. Se evita mediante:

- un *cuello de botella* latente suficientemente pequeño, que obliga a comprimir y a retener solo las regularidades de las piezas normales;
- variantes que rompen la copia trivial por construcción: *denoising autoencoder* (reconstruir a partir de una versión corrompida de la entrada) o *inpainting* (reconstruir regiones ocultas de la imagen);
- regularización del espacio latente y verificación en validación de que el error sobre piezas normales no se anule trivialmente.

De este modo el modelo solo puede reconstruir bien aquello que se parece a lo normal, y un defecto queda fuera de lo que sabe representar.

3.3 Puntaje de anomalía, umbral de decisión y caso de falla

Puntaje de anomalía. La imagen nueva se pasa por el autoencoder y se calcula el error de reconstrucción: un mapa de error por píxel $|\mathbf{x} - \hat{\mathbf{x}}|^2$ que se agrega a un escalar, por ejemplo tomando el máximo o el promedio de los K píxeles con mayor error, criterio más sensible a defectos pequeños y localizados que el promedio global. El mapa de error, además, localiza espacialmente el defecto para el inspector.

Criterio de decisión. Se calcula la distribución del puntaje sobre un conjunto de validación de piezas normales *no vistas en entrenamiento* y se fija el umbral en un percentil alto de esa distribución (por ejemplo, el percentil 95 a 99) o en $\mu + k\sigma$. La elección concreta es una decisión de costos: un umbral bajo genera más falsas alarmas (revisiones innecesarias)

y un umbral alto deja pasar defectos; en control de calidad usualmente se prefiere tolerar falsas alarmas antes que despachar piezas defectuosas.

Caso en que una pieza defectuosa sería considerada normal. El sistema falla cuando el defecto resulta fácil de reconstruir para el modelo. Por ejemplo:

- un defecto sutil y de baja frecuencia espacial (un leve cambio de tono, una deformación suave) que el autoencoder reconstruye casi a la perfección porque localmente se parece a texturas normales, quedando el error bajo el umbral;
- un defecto estructural o global (una perforación faltante, un componente en posición incorrecta) en el que cada parche local luce perfectamente normal: si el puntaje se basa en apariencia local, la pieza pasa aunque su configuración global sea inválida;
- de manera más general, un autoencoder con demasiada capacidad puede generalizar en exceso y reproducir (o “reparar”) patrones defectuosos que caen dentro de lo que alcanza a representar.

Esta es la limitación de fondo del enfoque: un error de reconstrucción bajo es solo un *proxy* de normalidad, no una garantía de ella.